

Async Operations

Section Review

Synchronous vs. Asynchronous Code

- Synchronous code *blocks* the current thread until a task is complete.
- Asynchronous code can transfer execution to another task while the first task waits on I/O.
- Python's built-in async event loop manages the transfer of control from task to task.
- Asynchronous code *can* lead to faster servers because the same thread can stay busier more of the time.

Making our Database Async

- The **aiosqlite** library enables async support for SQLite3.
- Other databases (Postgres, MySQL) have their own async drivers/libraries.
- We updated our database setup to include an **AsyncEngine**, an **AsyncSession**, and an async **get_session** dependency.

Async Syntax in FastAPI I

- Add the **async** keyword before **def** in an async function declaration.
- In the function body, use **await** before awaitable operations/methods.
- Not all libraries support async methods. Do **NOT** use blocking I/O code within an async route handler.

Async Syntax in FastAPI II

- If a route has IO/blocking code that does not support async, use a regular synchronous **def** function.
- If a route does not run any IO/blocking code, async or sync isn't too important. Choose one.

Async Syntax with SQLAlchemy ORM

- Methods like **get**, **exec**, **commit**, and **refresh** support the asynchronous syntax once we set up the async database session.
- The **add** method does not interact with the database, so it is synchronous.
- When SQLAlchemy saves a model to the database, all attributes are marked as expired. In async mode, call the **refresh** method to refetch and map the database record.